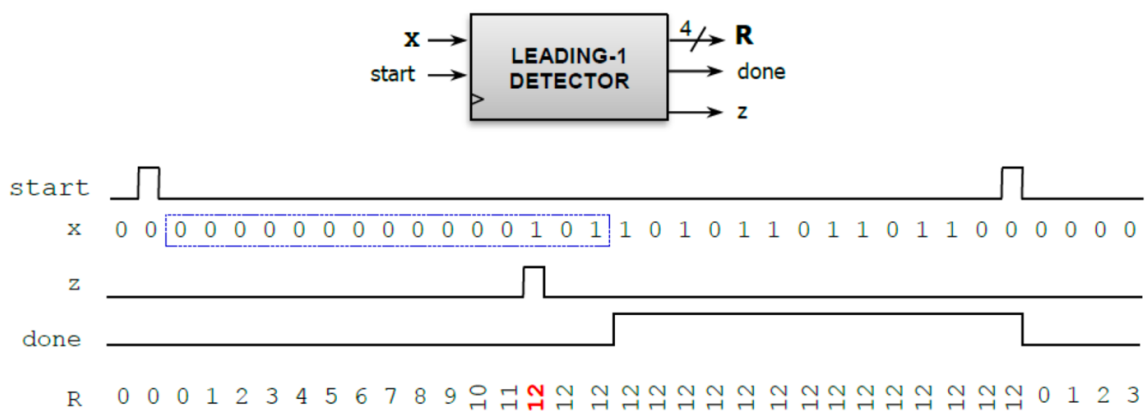


CAD-HW03-exe4

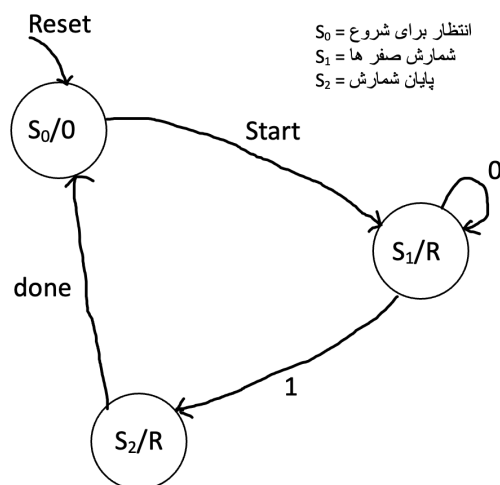
Hanie Vatani

14000122668035

برای طراحی یک شمارنده بیت های صفر یک رشته 15 بیتی، با ورودی های Start، X، Reset، Clk است. خروجی های این برنامه R (چهاربیتی، نشان دهنده تعداد صفر ها)، Z (اولین یک پیدا شد برای یک کلاک یک شود)، و done (با فعال شدن Start صفر شود و بعد از 15 کلاک یک شود تا نشان دهد دریافت داده 15 بیتی تمام شده است) هستند.



ماشین حالت:



کد برنامه :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity exe4 is
    port (
        Reset, Clk, Start: in std_logic;
        X: in BIT;
        Z, done: out std_logic;
        R: out std_logic_vector(3 downto 0) := (others => '0')
    );
end exe4;

architecture exe4 of exe4 is
    signal startPrv: std_logic := '0'; -- stores starts previous state
    signal bitCount: integer range 0 to 15 := 0; -- for counting bits
    signal zeroCount: integer range 0 to 15 := 0; --for counting 0s, there are maximum 15 zero bits in 15 bits
    signal ZH: std_logic; -- Z signal helper
    signal counting: boolean := false; -- shows that we count or not
    signal doneH: std_logic; -- done signal helper
    signal R2: std_logic_vector(3 downto 0) := (others => '0'); --R

begin
    process (Reset, Clk, Start)
    begin
        --default state
        ZH <= ZH;
        bitCount <= bitCount;
        zeroCount <= zeroCount;
        doneH <= doneH;
        counting <= counting;

        if (Reset = '1') then --reset
            bitCount <= 0;
            zeroCount <= 0;
            ZH <= '0';
            doneH <= '1';
            counting <= false;
        elsif (Clk'event and Clk = '1') then --checks X every 0.5ns for changes

            if (start = '1') then --start point
                startPrv <= '1';
            end if;

            if (Start = '0' and startPrv = '1') then --after start is 1 and then 0 done gets 0
                doneH <= '0';
                startPrv <= '0';
                bitCount <= 0;
                zeroCount <= 0;
                ZH <= '0';
                counting <= true;
            end if;
        end if;
    end process;
end architecture;
```

```

        if (doneH = '0') then --15 bits is not over
            if (counting = true) then --we didn't reached 1
                if (X = '0') then
                    zeroCount <= zeroCount + 1;
                else --reaching first 1
                    ZH <= '1';
                    counting <= false;
                end if;
            else --after reaching first 1
                ZH <= '0';
            end if;
            bitCount <= bitCount + 1; --bit counts goes up either way
        end if;
    end if;

    if (bitCount = 15) then --end of the 15 bits
        bitCount <= 0;
        zeroCount <= 0;
        ZH <= '0';
        doneH <= '1';
        counting <= false;
    end if;
end process;
R2 <= std_logic_vector(to_unsigned(zeroCount, 4));
done <= doneH;
Z <= ZH;
R <= R2;
end exe4;

```

* برای این پروژه کلاک اصلی X در نظر گرفته شده اما برای تداوم چک کردن از Clk با تکرار چک در هر 0.5ns استفاده شده. یعنی برنامه هر 1ns تغییر پیدا میکند اما هر 0.5ns برنامه برای تغییر چک می‌شود. پس وقتی گفته شده Z برای یک کلاک 1 شود، کلاک 1ns در نظر گرفته شده.

سپس کد را compile کرده و فایل test bench برای شبیه سازی می‌سازیم. و به آن کد زیر را اضافه می‌کنیم:

```

Reset <= '1', '0' after 0.4ns;

Clk <= '0', '1' after 0.5ns, '0' after 1ns, '1' after 1.5ns, '0' after 2ns, '1' after 2.5ns, '0' after 3ns, '1' after 3.5ns,
'0' after 4ns, '1' after 4.5ns, '0' after 5ns, '1' after 5.5ns, '0' after 6ns, '1' after 6.5ns, '0' after 7ns, '1' after 7.5ns,
'0' after 8ns, '1' after 8.5ns, '0' after 9ns, '1' after 9.5ns, '0' after 10ns, '1' after 10.5ns, '0' after 11ns, '1' after 11.5ns,
'0' after 12ns, '1' after 12.5ns, '0' after 13ns, '1' after 13.5ns, '0' after 14ns, '1' after 14.5ns, '0' after 15ns, '1' after 15.5ns,
'0' after 16ns, '1' after 16.5ns, '0' after 17ns, '1' after 17.5ns, '0' after 18ns, '1' after 18.5ns, '0' after 19ns, '1' after 19.5ns,
'0' after 20ns, '1' after 20.5ns, '0' after 21ns, '1' after 21.5ns, '0' after 22ns, '1' after 22.5ns, '0' after 23ns, '1' after 23.5ns,
'0' after 24ns, '1' after 24.5ns, '0' after 25ns, '1' after 25.5ns, '0' after 26ns, '1' after 26.5ns, '0' after 27ns, '1' after 27.5ns,
'0' after 28ns, '1' after 28.5ns, '0' after 29ns, '1' after 29.5ns, '0' after 30ns, '1' after 30.5ns, '0' after 31ns, '1' after 31.5ns,
'0' after 32ns, '1' after 32.5ns, '0' after 33ns, '1' after 33.5ns, '0' after 34ns, '1' after 34.5ns, '0' after 35ns;

Start <= '0', '1' after 0.5ns, '0' after 1.5ns, '1' after 19.5ns, '0' after 20.5ns;

X <= '0', '0' after 1ns, '0' after 2ns, '0' after 3ns, '0' after 4ns, '0' after 5ns, '0' after 6ns, '0' after 7ns,
'0' after 8ns, '1' after 9ns, '0' after 10ns, '1' after 11ns, '0' after 12ns, '1' after 13ns, '0' after 14ns, '0' after 15ns,
'1' after 16ns, '1' after 17ns, '0' after 18ns, '0' after 19ns, '0' after 20ns, '0' after 21ns, '0' after 22ns, '0' after 23ns,
'0' after 24ns, '1' after 25ns, '0' after 26ns, '1' after 27ns, '1' after 28ns, '1' after 29ns, '0' after 30ns, '1' after 31ns,
'0' after 32ns, '1' after 33ns, '1' after 34ns, '0' after 35ns;

```

بعد از compile کردن فایل test bench آن را شبیه سازی می‌کنیم. برای این کار فایل test bench را excute می‌کنیم. و سپس برنامه را run می‌کنیم.

نتیجه ی شبیه سازی:

